

Flask & RESTful APIs

From Basic to Advanced – A Project Development Perspective

Dr. Anilkumar V. Brahmane

Computer Engineering Department

Sanjivani College of Engineering

- ❏ Flask helps us create a web server/API in Python, allowing other applications to communicate with our Python app – including ML and AI models – over the internet.





What is an API? The Restaurant Analogy

Real-Life Analogy

In a restaurant, the **customer** never enters the kitchen directly. The **waiter** acts as the interface – taking requests and delivering responses.

Customer → Waiter → Kitchen → Food
→ Customer

In Software

Similarly, the **frontend** never directly touches the backend. The **API** acts as the communication interface.

Frontend → API → Backend → Database
/ ML Model

📄 API = Application Programming Interface – a messenger between two software applications.

— Dr. Anilkumar V. Brahmane | Computer Engineering Dept., Sanjivani College of Engineering

Why Flask for ML Projects?

Flask is a **lightweight Python web framework** for building web apps, REST APIs, and ML/AI backends. Without Flask, exposing a Python ML model to the web requires much more low-level work.

The Problem

After training a model with `model.fit(X_train, y_train)`, your website, Android app, or IoT device **cannot call a Python variable directly**.

The Solution

Wrap the model in a **Flask REST API**. Now any application — web, mobile, IoT — can send data and receive predictions via HTTP.

Project Value

Students can take any ML/AI model and **expose it as an API** that a web or mobile frontend can consume — a core industry skill.

— Dr. Anilkumar V. Brahmane | Computer Engineering Dept., Sanjivani College of Engineering

Your First Flask Application

Install & Run

```
pip install flask
```

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, World!"

if __name__ == "__main__":
    app.run(debug=True)
```

Run with `python app.py`. Visit `http://127.0.0.1:5000` to see **Hello, World!**

Understanding Routes

A **route** tells Flask: "When someone requests this URL, execute this function."

```
@app.route("/students")
def students():
    return "Student Page"
```

You can define **multiple routes** for `/`, `/about`, `/students`, etc. — each mapped to its own function.

❏ Routes are the building blocks of every Flask project endpoint.

HTTP Methods, JSON & REST

The Four HTTP Methods (CRUD)

Method	CRUD	Purpose
GET	Read	Fetch data
POST	Create	Send/create data
PUT	Update	Modify existing data
DELETE	Delete	Remove data

REST Resource Design

```
GET /students # All students
POST /students # Add student
GET /students/101 # Student 101
PUT /students/101 # Update 101
DELETE /students/101 # Delete 101
```

JSON — The Common Language

JSON (JavaScript Object Notation) is the standard data format for REST APIs. React, Flask, and Python all speak JSON.

```
{
  "name": "Rahul",
  "age": 20,
  "branch": "CE"
}
```

Flask POST Example

```
data = request.get_json()
name = data["name"]
return jsonify({
  "message": "Student added",
  "name": name
}), 201
```

Flask + ML Model: The /predict API

This is the most important pattern for AI/ML student projects. Save your trained model, load it in Flask, and expose a /predict endpoint.

Save & Load Model

```
import joblib
# Save
joblib.dump(model, "student_model.pkl")
# Load in Flask
model = joblib.load("student_model.pkl")
```

The /predict Endpoint

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    study_hours = data["study_hours"]
    attendance = data["attendance"]
    prev_marks = data["previous_marks"]
    pred = model.predict(
        [[study_hours, attendance, prev_marks]]
    )
    result = "PASS" if pred[0]==1 else "FAIL"
    return jsonify({"prediction": result})
```

How It Works



Client sends {"study_hours":6, "attendance":85, "previous_marks":70}
and receives {"prediction":"PASS"}.

Advanced Flask Concepts for Projects

URL & Query Parameters

Use `/students/<int:student_id>` for path params, or `/students?branch=CE&year=4` with `request.args.get()` for query params.

HTTP Status Codes

Return 200 (Success), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), 500 (Server Error) for professional APIs.

Input Validation & Error Handling

Always validate inputs before passing to the model. If `study_hours` is missing or `-50`, return a 400 error with a meaningful message – never crash silently.

CORS for React Integration

Install `flask-cors` and use `CORS(app)` so your React frontend at `localhost:3000` can call your Flask API at `localhost:5000` without browser blocks.

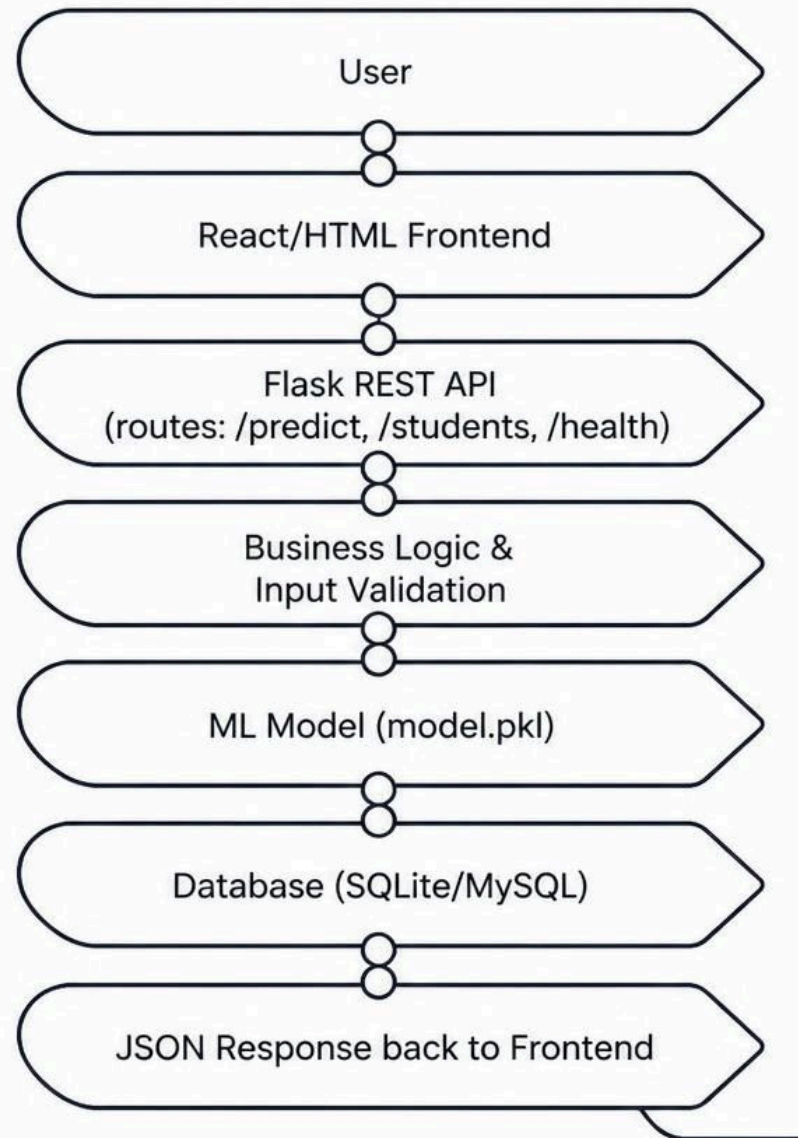
API Authentication

For private ML APIs, implement API keys or JWT tokens. Students can demonstrate the concept with a simple API-key check; production systems require stronger security.

API Versioning

Use `/api/v1/predict` so existing apps aren't broken when you release `/api/v2/predict` – a critical industry practice.

Complete Project Architecture



This architecture separates concerns: the frontend handles UI, Flask handles routing and validation, the ML model handles inference, and the database stores predictions. Each component has a **single responsibility** — far better than putting everything in one `main.py`.

Student Project Ideas Using Flask + ML



Student Performance Prediction

Input: study hours, attendance, marks → Flask → ML Model → Pass/Fail



House Price Prediction

Input: house details → Flask API → Regression Model → Predicted Price



Disease Risk Prediction

Patient parameters → Flask → ML Model → Risk score (research/decision-support prototype only)



Flask + LLM / RAG Chatbot

User question → Flask API → RAG + Vector DB + LLM → Answer. Ideal for Agentic AI projects using Gemini or OpenAI-compatible APIs.

Other ideas: Loan Approval, Fake News Classification, Sentiment Analysis, Image Classification, Crop Disease Detection (CNN).

— Dr. Anilkumar V. Brahmane / Computer Engineering Dept., Sanjivani College of Engineering

Learning Roadmap & Key Terms

Three-Level Learning Path

01

Level 1 — Flask Basics

Installation, routes, GET/POST, JSON, request, jsonify, status codes

02

Level 2 — ML Integration

Train → Serialize → Load → /predict API → Input validation → Error handling

03

Level 3 — Industry Project

React frontend + Flask + DB + Auth + CORS + Versioning + Postman + Docker + Cloud + Monitoring

10 Terms Every Student Must Know

Term	Meaning
Flask	Python web framework
API	Communication interface
REST	Web API design style
Route	URL handled by app
GET / POST	Read / Send data
JSON	Common data format
Endpoint	Specific API URL
Serialization	Save trained model
Inference	Model makes prediction

📌 Practical sequence: Scikit-learn → joblib → Flask → REST API → /predict → Postman → React → Full ML Web App → Deployment

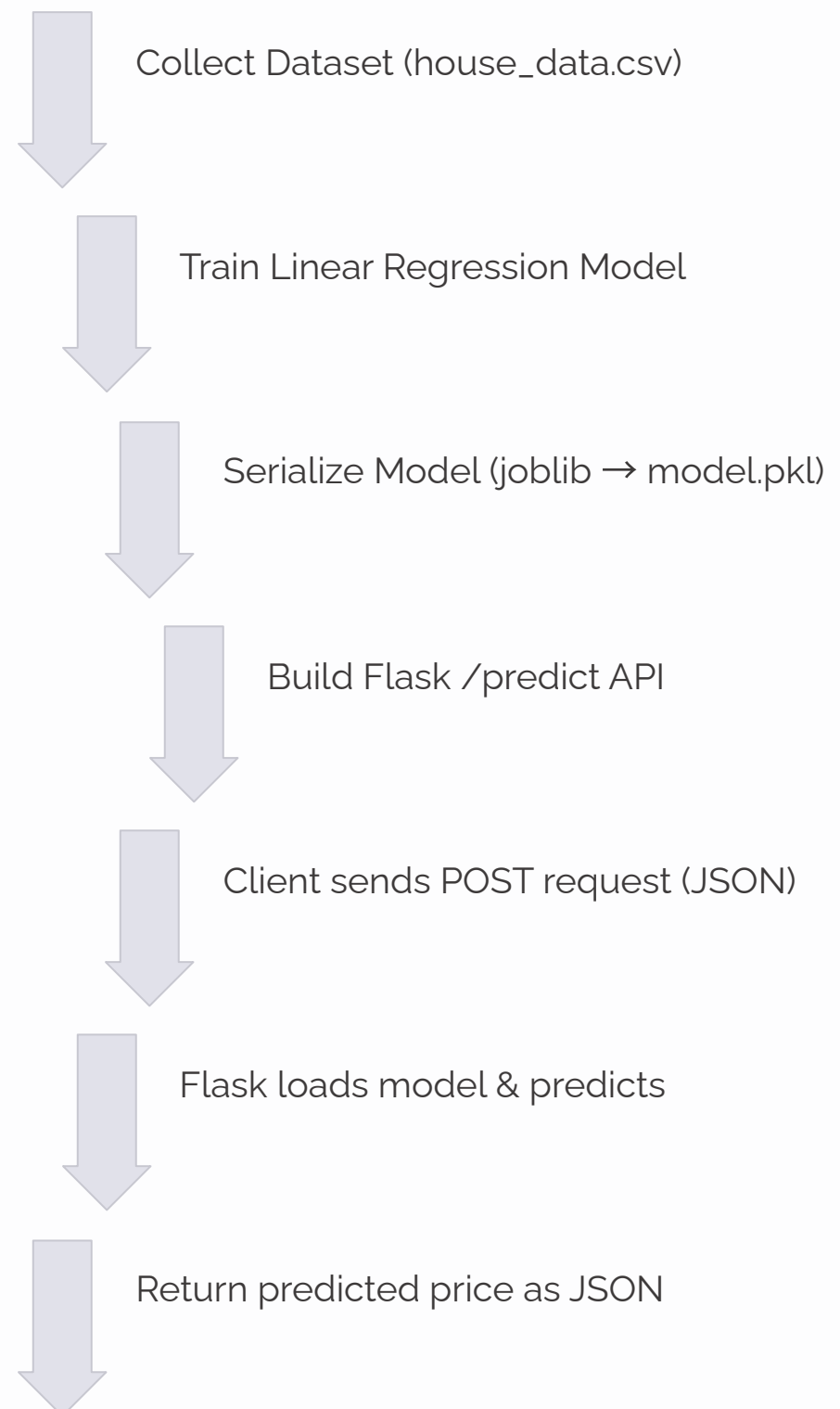
Project 1: House Price Prediction — Overview & Architecture

A complete Flask REST API that takes house features as input and returns a predicted price using a trained ML regression model.

Project Features

- Input: Area (sq ft), Bedrooms, Bathrooms, Location score
- Output: Predicted house price (₹)
- Model: Linear Regression (scikit-learn)
- API: Flask REST endpoint /predict
- Format: JSON request & response

Flowchart: End-to-End Flow



House Price Prediction — Step 1: Dataset & Model Training

Before building the API, we train a Linear Regression model on historical house data and save it using joblib.

Step 1A — Create / Load Dataset

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import joblib

# Sample dataset
data = {
    "area": [1000, 1500, 2000, 2500, 3000],
    "bedrooms": [2, 3, 3, 4, 5],
    "bathrooms": [1, 2, 2, 3, 3],
    "location": [7, 8, 9, 8, 10],
    "price": [3500000, 5000000, 7000000,
              8500000, 12000000]
}
df = pd.DataFrame(data)
```

Each row = one house. 'price' is the target (label) column.

Step 1B — Train & Save Model

```
X = df[["area", "bedrooms", "bathrooms", "location"]]
y = df["price"]

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2,
                    random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

# Save model to disk
joblib.dump(model, "house_model.pkl")
print("Model saved as house_model.pkl")
```

What Happens at Each Line?

01

Import Libraries

pandas for data, sklearn for ML, joblib for saving

02

Create DataFrame

structured table of house features + prices

03

Split Features & Target

X = inputs, y = price to predict

04

train_test_split

80% training data, 20% for testing

05

model.fit()

model learns the relationship between features and price

06

joblib.dump()

serializes the trained model to house_model.pkl on disk

📄 joblib is preferred over pickle for scikit-learn models — it handles large numpy arrays more efficiently.

House Price Prediction — Step 2: Building the Flask API

Now we build the Flask app that loads the saved model and exposes a /predict endpoint for house price prediction.

app.py — Complete Flask API

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)

# Load model once at startup
model = joblib.load("house_model.pkl")

@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "House Price Prediction API",
        "version": "1.0",
        "endpoints": ["/predict", "/health"]
    })

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "running"}), 200

@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json()

        # Extract & validate inputs
        area    = data["area"]
        bedrooms = data["bedrooms"]
        bathrooms = data["bathrooms"]
        location = data["location"]

        if area <= 0 or bedrooms <= 0:
            return jsonify({
                "error": "Invalid input values"
            }), 400

        # Prepare input for model
        features = np.array([[area, bedrooms,
                               bathrooms, location]])
        price = model.predict(features)[0]

        return jsonify({
            "predicted_price": round(float(price), 2),
            "currency": "INR"
        }), 200

    except KeyError as e:
        return jsonify({
            "error": f"Missing field: {str(e)}"
        }), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)
```

Line-by-Line Explanation

- joblib.load() at startup** — Model is loaded ONCE when Flask starts, not on every request. This is critical for performance.
- GET /health** — A health-check endpoint. Used by deployment platforms (Docker, cloud) to verify the API is alive.
- request.get_json()** — Parses the incoming JSON body from the POST request into a Python dictionary.
- Input Validation** — Checks for invalid values ($\text{area} \leq 0$) before passing to model. Returns HTTP 400 if invalid.
- np.array([[...]])** — Model expects a 2D array. Double brackets create shape (1, 4) — one sample, four features.
- model.predict()[0]** — Returns an array; [0] extracts the single predicted value.
- try/except KeyError** — Catches missing JSON fields and returns a clear 400 error message.
- jsonify()** — Converts Python dict to proper JSON HTTP response with correct Content-Type header.

House Price Prediction — Step 3: Testing the API

Test the /predict endpoint using Postman or Python's requests library. Understand the request/response cycle.

Method 1: Using Postman

- 01

- Open Postman → New Request
- 02

- Set method to POST, URL: http://127.0.0.1:5000/predict
- 03

- Go to Body → raw → JSON
- 04

- Paste the JSON body (shown below)
- 05

- Click Send → See predicted price in response

```
{
  "area": 2000,
  "bedrooms": 3,
  "bathrooms": 2,
  "location": 9
}
```

Expected JSON response:

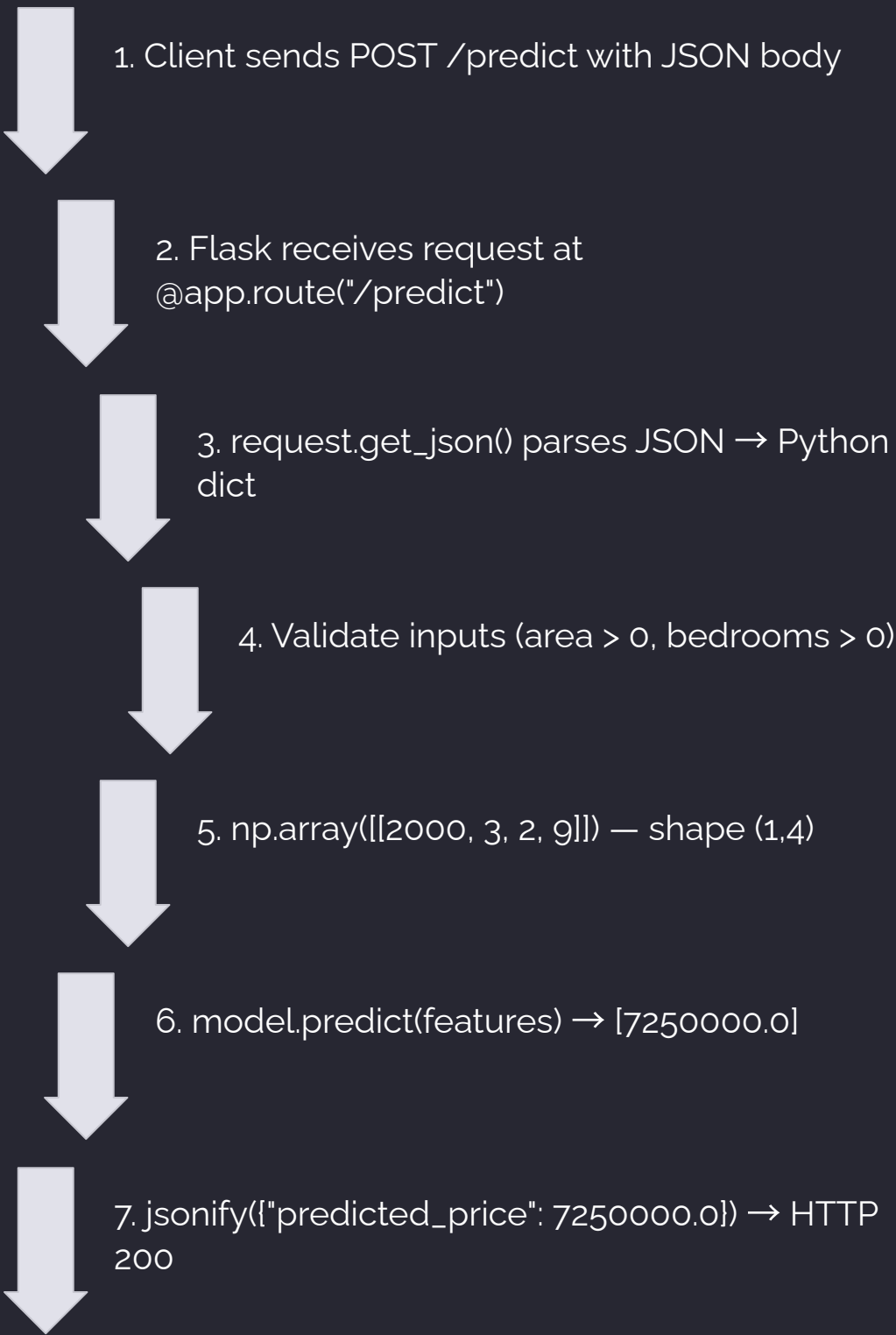
```
{
  "predicted_price": 7250000.00,
  "currency": "INR"
}
```

Method 2: Using Python requests

```
import requests

url = "http://127.0.0.1:5000/predict"
payload = {
  "area": 2000,
  "bedrooms": 3,
  "bathrooms": 2,
  "location": 9
}
response = requests.post(url, json=payload)
print(response.json())
# Output: {'predicted_price': 7250000.0,
#         'currency': 'INR'}
```

Request → Response Flowchart



Status Code	Scenario	Meaning
200	Valid input, prediction made	Success
400	Missing field or invalid value	Bad Request
500	Unexpected server error	Internal Error

Project 2: Student Performance Prediction — Overview & Architecture

A Flask REST API that predicts whether a student will PASS or FAIL based on study hours, attendance, and previous marks using a Logistic Regression classifier.

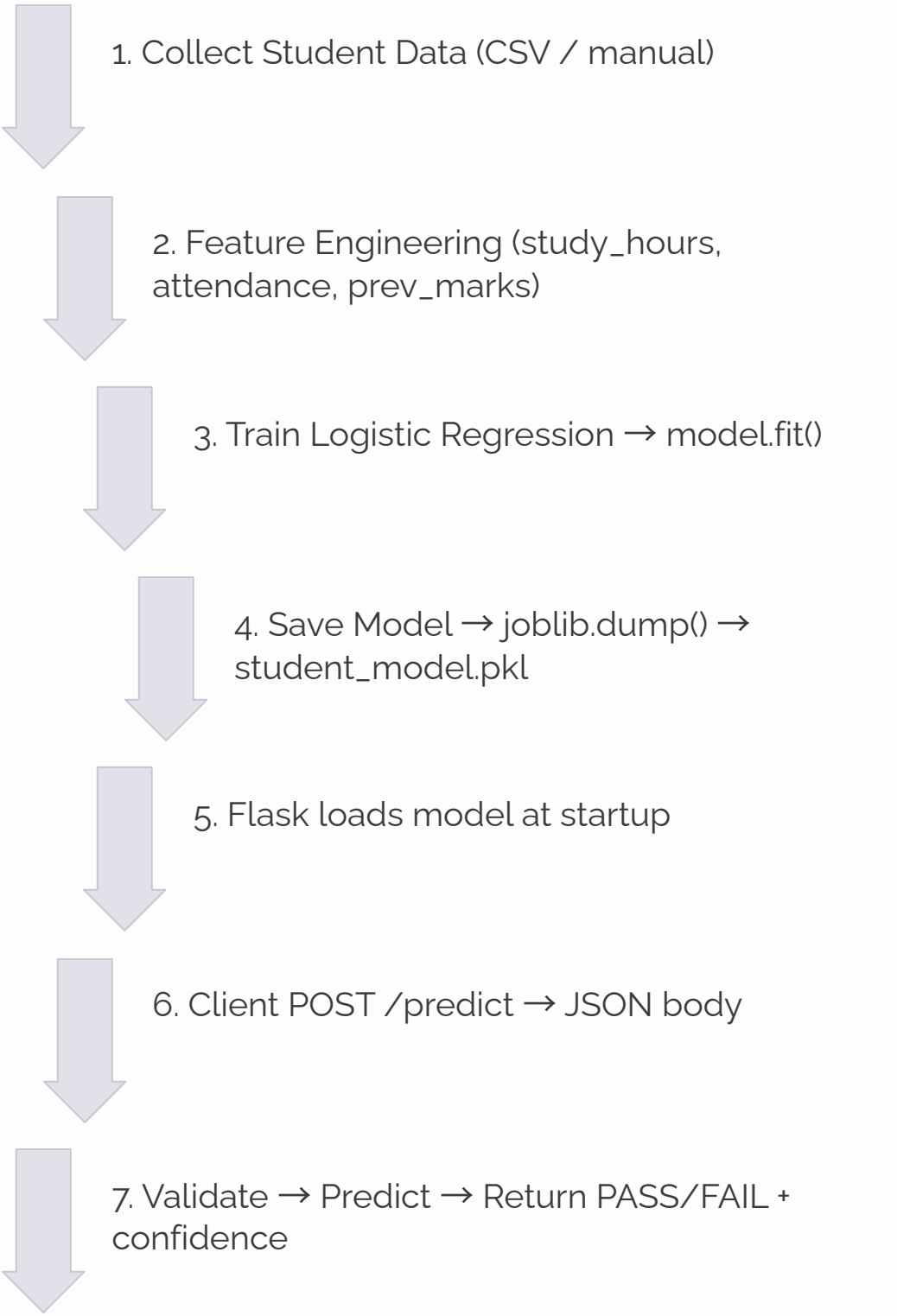
Project Features

- Input: Study hours, Attendance %, Previous marks
- Output: PASS or FAIL prediction + confidence %
- Model: Logistic Regression (scikit-learn)
- API Endpoints: /predict, /health, /batch-predict
- Format: JSON request & response

Project File Structure

```
student_project/  
├── train_model.py    # Train & save model  
├── app.py            # Flask REST API  
├── student_model.pkl # Saved model  
├── requirements.txt  # Dependencies  
└── test_api.py       # API test script
```

End-to-End Architecture Flowchart



📌 This is the exact pattern used in industry AI products — the model is always separate from the API layer.

Student Performance — Step 1: Training the Classifier

We train a Logistic Regression model that classifies students as PASS (1) or FAIL (0) based on three features.

train_model.py — Full Code

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import joblib

# Step 1: Dataset
data = {
    "study_hours": [2,4,6,8,3,7,5,1,9,6],
    "attendance": [50,65,80,90,55,85,75,40,95,78],
    "prev_marks": [40,55,70,85,45,80,65,35,92,72],
    "result": [0, 0, 1, 1, 0, 1, 1, 0, 1, 1]
}
df = pd.DataFrame(data)

# Step 2: Features & Target
X = df[["study_hours","attendance","prev_marks"]]
y = df["result"] # 0=FAIL, 1=PASS

# Step 3: Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Split & Train
X_train, X_test, y_train, y_test = \
    train_test_split(X_scaled, y,
                    test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

# Step 5: Evaluate
preds = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test,preds)*100:.1f}%")

# Step 6: Save both model AND scaler
joblib.dump(model, "student_model.pkl")
joblib.dump(scaler, "student_scaler.pkl")
print("Model and scaler saved!")
```

Why Each Step Matters

result column: 0 or 1 — Logistic Regression is a binary classifier. 0 = FAIL, 1 = PASS. This is the label the model learns to predict.

StandardScaler — Normalizes features so study_hours (range 1–9) doesn't dominate attendance (range 40–95). Always scale for Logistic Regression.

scaler.fit_transform(X) — fit() learns mean & std from training data. transform() applies normalization. Never fit on test data — data leakage!

Save the scaler too! — Critical: the same scaler used during training MUST be used during prediction. Save it with joblib alongside the model.

accuracy_score — Measures what % of test predictions were correct. For a student project, 80%+ is a good baseline.

❏ Always save the scaler! If you scale during training but not during prediction, your model will give completely wrong results.

Student Performance — Step 2: Building the Flask API

The Flask app loads both the model and scaler, validates inputs, scales them identically to training, and returns PASS/FAIL with a confidence percentage.

app.py — Complete Flask API

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)

# Load model & scaler at startup
model = joblib.load("student_model.pkl")
scaler = joblib.load("student_scaler.pkl")

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "running"}), 200

@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json()

        study_hours = float(data["study_hours"])
        attendance = float(data["attendance"])
        prev_marks = float(data["prev_marks"])

        # Validate ranges
        if not (0 <= study_hours <= 24):
            return jsonify({
                "error": "study_hours must be 0–24"
            }), 400
        if not (0 <= attendance <= 100):
            return jsonify({
                "error": "attendance must be 0–100"
            }), 400
        if not (0 <= prev_marks <= 100):
            return jsonify({
                "error": "prev_marks must be 0–100"
            }), 400

        # Scale input (same as training)
        features = np.array([[study_hours,
                               attendance,
                               prev_marks]])
        features_scaled = scaler.transform(features)

        # Predict class & probability
        prediction = model.predict(features_scaled)[0]
        probability = model.predict_proba(
            features_scaled)[0]

        result = "PASS" if prediction == 1 else "FAIL"
        confidence = round(float(max(probability))*100, 2)

        return jsonify({
            "prediction": result,
            "confidence": f"{confidence}%",
            "study_hours": study_hours,
            "attendance": attendance,
            "prev_marks": prev_marks
        }), 200

    except KeyError as e:
        return jsonify({
            "error": f"Missing field: {str(e)}"
        }), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)
```

Step-by-Step Execution Flow

01

1. Load model & scaler

Both loaded once at startup for performance

02

2. Parse JSON

request.get_json() converts body to Python dict

03

3. Type casting

float() ensures numeric types even if client sends strings

04

4. Range validation

Checks realistic bounds (attendance 0–100, marks 0–100)

05

5. scaler.transform()

Applies the SAME normalization used during training

06

6. model.predict()

Returns 0 (FAIL) or 1 (PASS)

07

7. model.predict_proba()

Returns [prob_fail, prob_pass] — e.g., [0.12, 0.88]

08

8. max(probability)

Picks the higher confidence value (88% in example)

09

9. Return JSON

prediction + confidence + echoed inputs for transparency

📌 predict_proba() is a powerful feature of Logistic Regression — it tells you HOW confident the model is, not just what it predicts.

Student Performance — Step 3: Testing & Batch Prediction

Test the API with single and batch predictions. The /batch-predict endpoint allows multiple students to be evaluated in one API call.

Single Prediction — Postman / requests

```
POST http://127.0.0.1:5000/predict
Content-Type: application/json
```

```
{
  "study_hours": 6,
  "attendance": 80,
  "prev_marks": 72
}
```

```
{
  "prediction": "PASS",
  "confidence": "87.43%",
  "study_hours": 6.0,
  "attendance": 80.0,
  "prev_marks": 72.0
}
```

Bonus: /batch-predict Endpoint

```
@app.route("/batch-predict", methods=["POST"])
def batch_predict():
    try:
        students = request.get_json()["students"]
        results = []
        for s in students:
            feat = np.array([[
                float(s["study_hours"]),
                float(s["attendance"]),
                float(s["prev_marks"])
            ]])
            feat_scaled = scaler.transform(feat)
            pred = model.predict(feat_scaled)[0]
            proba = model.predict_proba(feat_scaled)[0]
            results.append({
                "name": s.get("name", "Unknown"),
                "prediction": "PASS" if pred==1 else "FAIL",
                "confidence": f"{round(float(max(proba))*100,2)}%"
            })
        return jsonify({"results": results}), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Batch Request & Response Example

```
POST /batch-predict
{
  "students": [
    {"name": "Rahul", "study_hours": 7,
     "attendance": 85, "prev_marks": 78},
    {"name": "Priya", "study_hours": 2,
     "attendance": 45, "prev_marks": 38},
    {"name": "Amit", "study_hours": 5,
     "attendance": 72, "prev_marks": 65}
  ]
}

{
  "results": [
    {"name": "Rahul", "prediction": "PASS",
     "confidence": "91.2%"},
    {"name": "Priya", "prediction": "FAIL",
     "confidence": "88.7%"},
    {"name": "Amit", "prediction": "PASS",
     "confidence": "76.4%"}
  ]
}
```

Key Takeaways

Single /predict — One student at a time. Best for real-time frontend forms.

/batch-predict — Multiple students in one call. Best for bulk processing or reports.

confidence % — predict_proba() makes your API far more useful than a simple PASS/FAIL.

Project Comparison: House Price vs Student Performance

Both projects follow the same Flask + ML pattern. Understanding the differences helps you adapt this architecture to any ML project.

Aspect	House Price Prediction	Student Performance	Key Difference
ML Task	Regression (continuous output)	Classification (binary output)	Regression predicts a number; Classification predicts a category
Model	LinearRegression	LogisticRegression	Different algorithms for different output types
Output	Predicted price (₹7,25,000)	PASS / FAIL label	Number vs category
Confidence	Not applicable	predict_proba() → 87%	Classification models give probability scores
Scaling	Not required (optional)	StandardScaler (required)	Logistic Regression is sensitive to feature scale
Saved files	house_model.pkl	student_model.pkl + student_scaler.pkl	Always save scaler if used during training
Validation	area > 0, bedrooms > 0	0≤attendance≤100, 0≤marks≤100	Validate based on real-world domain constraints
HTTP Status	200 / 400 / 500	200 / 400 / 500	Same REST conventions for both projects

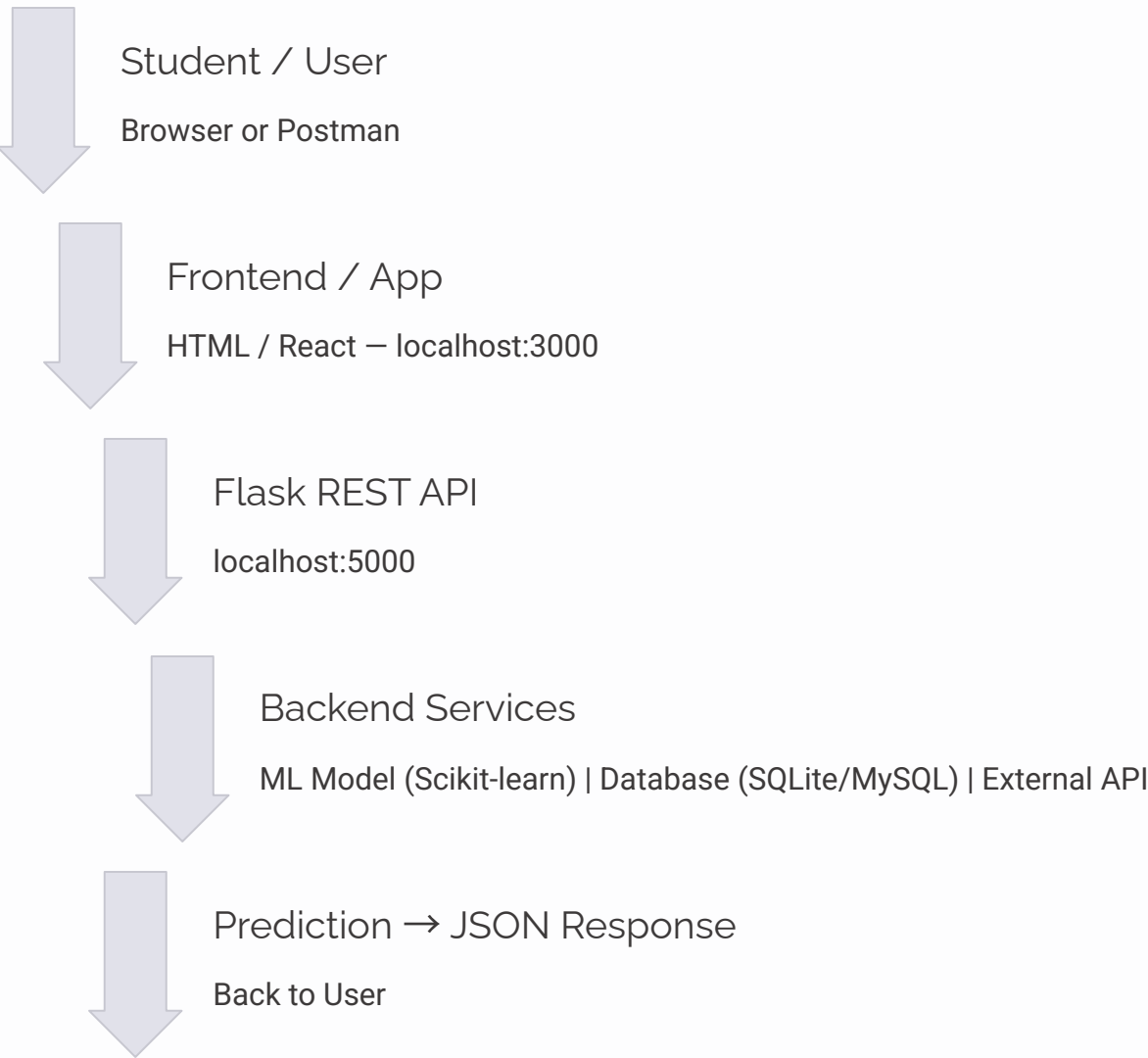
Universal Pattern – Train → Serialize → Load in Flask → Validate → Predict → Return JSON. This works for ANY ML model.

Regression Projects – House price, salary prediction, stock price, crop yield. Output is always a number.

Classification Projects – Student pass/fail, disease risk, spam detection, sentiment. Output is always a category.

Python Libraries for Flask & RESTful APIs — Overview

When building ML projects with Flask, we use several Python libraries together. Each library has a specific role in the pipeline.



Library	Main Purpose
Flask	Create web application / REST API
request	Receive data from client (JSON body)
jsonify	Send JSON response back to client
flask-cors	Connect frontend and backend (cross-origin)
joblib	Save and load trained ML models
pickle	Serialize Python objects to disk
requests	Call APIs from Python programs
Flask-RESTful	Build REST APIs using class-based resources
pydantic	Validate and type-check incoming data

📌 One-line rule for students: Flask creates the API → request receives data → joblib loads the model → model.predict() runs → jsonify sends the result.

Library 1: Flask — Creating the Web API

Flask is a lightweight Python web framework. It helps us create web applications, REST APIs, ML model APIs, and backend applications.

Install & Simple Example

```
pip install flask

from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Students!"

if __name__ == "__main__":
    app.run(debug=True)
```

Line-by-Line Breakdown

01

from flask import Flask

Import the Flask class from the flask package

02

app = Flask(__name__)

Create a Flask application instance. __name__ tells Flask where to find resources.

03

@app.route("/")

A decorator that maps the URL "/" to the function below it

04

def home(): return "Hello Students!"

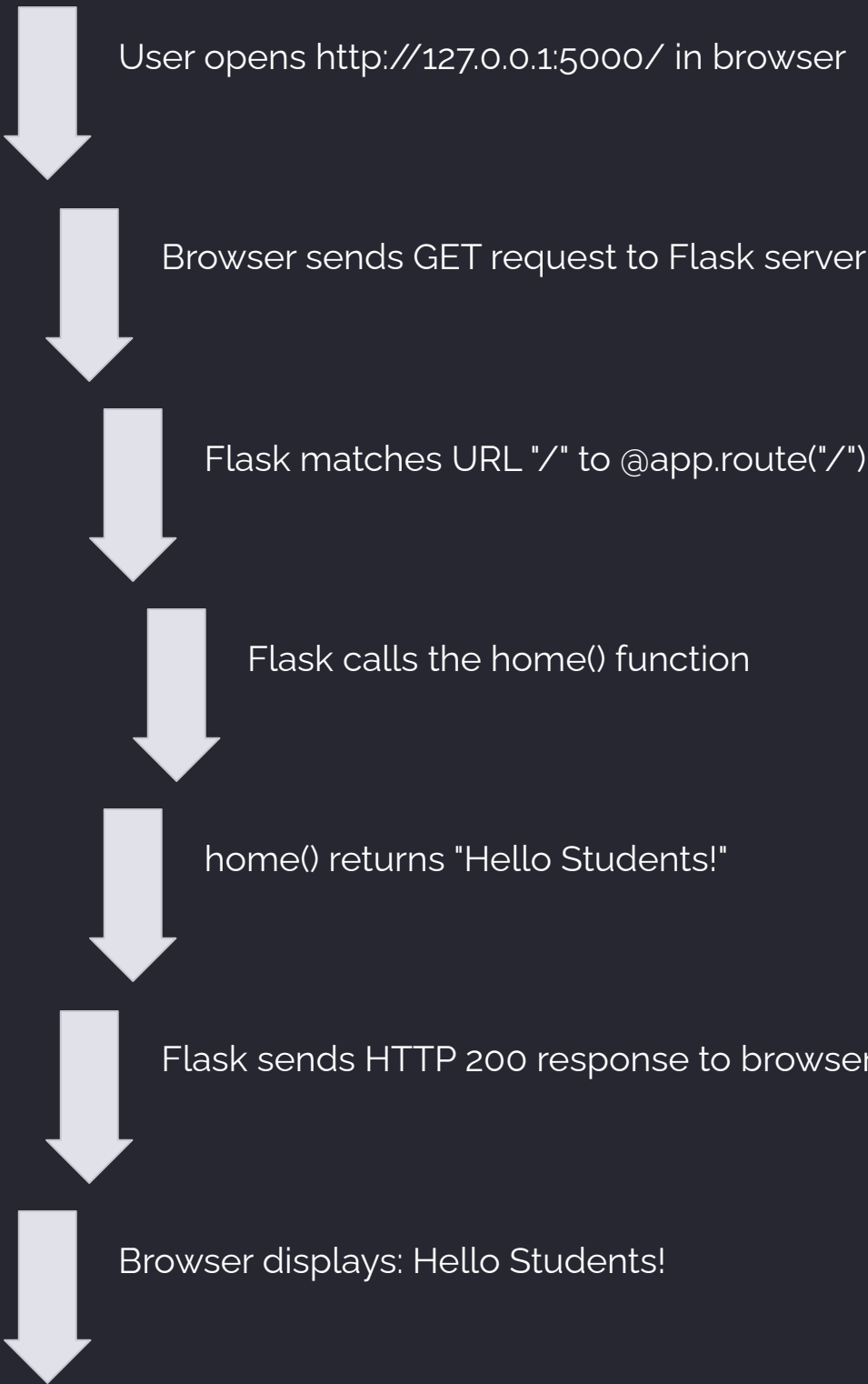
When user visits http://127.0.0.1:5000/, Flask runs this function and displays the return value

05

app.run(debug=True)

Starts the development server. debug=True auto-reloads on code changes.

How Flask Handles a Request



Flask Route Patterns

```
# Basic route
@app.route("/")

# Route with variable
@app.route("/student/<int:student_id>")

# Route with HTTP method
@app.route("/predict", methods=["POST"])

# Multiple methods
@app.route("/data", methods=["GET", "POST"])
```

❏ Flask is called 'micro' framework because it has no database layer or form validation built-in — you add only what you need. This makes it perfect for ML APIs.

Library 2 & 3: request & jsonify — Receive & Send Data

request reads incoming data from the client. jsonify converts a Python dictionary into a proper JSON HTTP response. These two always work together.

request — Receiving Client Data

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/predict", methods=["POST"])
def predict():

    # Read JSON body sent by client
    data = request.get_json()

    study_hours = data["study_hours"]
    attendance = data["attendance"]

    return "Data received!"
```

What request Can Read

Method	What it reads
request.get_json()	JSON body from POST request → Python dict
request.args.get("name")	URL query params: /student? name=Rahul
request.form.get("email")	HTML form data (multipart)
request.method	Returns "GET", "POST", "PUT", "DELETE"
request.headers	HTTP headers sent by client

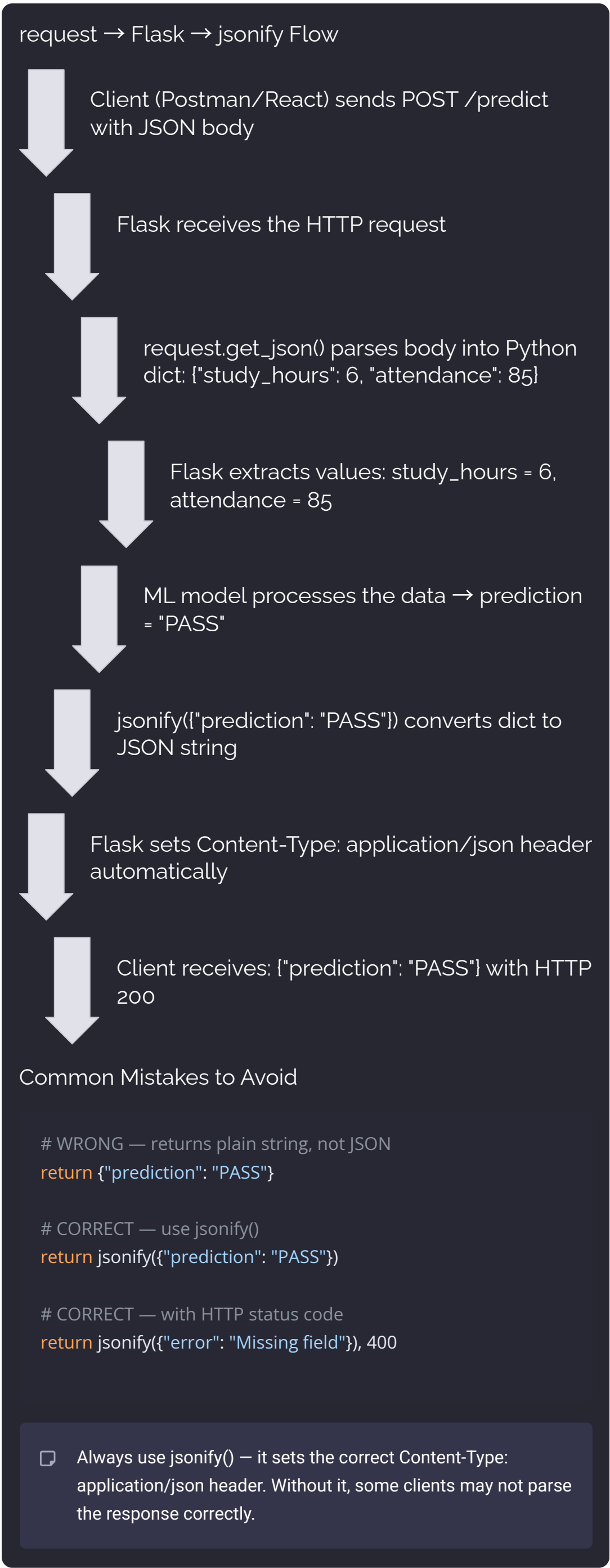
jsonify — Sending JSON Response

```
from flask import Flask, request, jsonify

@app.route("/student")
def student():

    return jsonify({
        "name": "Rahul",
        "marks": 85,
        "result": "PASS"
    })
```

API returns: { "name": "Rahul", "marks": 85, "result": "PASS" }



Library 4: flask-cors — Connecting Frontend & Backend

CORS (Cross-Origin Resource Sharing) is a browser security rule. When your React frontend (localhost:3000) calls your Flask API (localhost:5000), the browser blocks it by default. flask-cors fixes this.

Why CORS Happens

Different Origins

React runs on localhost:3000. Flask runs on localhost:5000. Different ports = different origins = browser blocks the request.

Browser Security Rule

Browsers enforce the Same-Origin Policy. A page from origin A cannot call an API on origin B without permission.

CORS Header Fix

The server must send: Access-Control-Allow-Origin: * in the response header to grant permission.

Install & Use flask-cors

pip install flask-cors

```
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)

# Allow ALL origins (development)
CORS(app)

# Allow specific origin only (production)
# CORS(app, origins=["http://localhost:3000"])

@app.route("/predict", methods=["POST"])
def predict():
    return jsonify({"prediction": "PASS"})

if __name__ == "__main__":
    app.run(debug=True)
```

Without CORS vs With CORS

Scenario	Without CORS	With CORS
React calls Flask API	Browser blocks: CORS error	Request succeeds
Response header	No Access-Control header	Access-Control-Allow-Origin: *
API works in Postman?	Yes (Postman ignores CORS)	Yes
API works in browser?	No — blocked	Yes — allowed
Error message	"CORS policy blocked"	No error

CORS Flow Diagram

React (localhost:3000) sends POST to Flask (localhost:5000)

Browser checks: same origin? NO → sends preflight OPTIONS request

Without CORS: Flask returns no Access-Control header → Browser blocks response

With CORS(app): Flask adds Access-Control-Allow-Origin: * to every response

Browser sees the header → allows the response → React receives data

In production, always restrict CORS to your specific frontend domain: CORS(app, origins=['https://yourapp.com']). Never use * in production.

Library 5 & 6: joblib & pickle — Saving & Loading ML Models

After training an ML model, we must save it to disk so Flask can load and reuse it without retraining every time the server starts.

joblib — Preferred for Scikit-learn

```
import joblib
from sklearn.linear_model import LogisticRegression

# --- TRAINING PHASE (train_model.py) ---
model = LogisticRegression()
model.fit(X_train, y_train)

# Save model to disk
joblib.dump(model, "student_model.pkl")
print("Model saved!")

# --- FLASK API PHASE (app.py) ---
# Load model once at startup
model = joblib.load("student_model.pkl")

# Now use it for prediction
prediction = model.predict([[6, 85, 72]])
print(prediction) # ['PASS']
```

pickle — Python's Built-in Serializer

```
import pickle

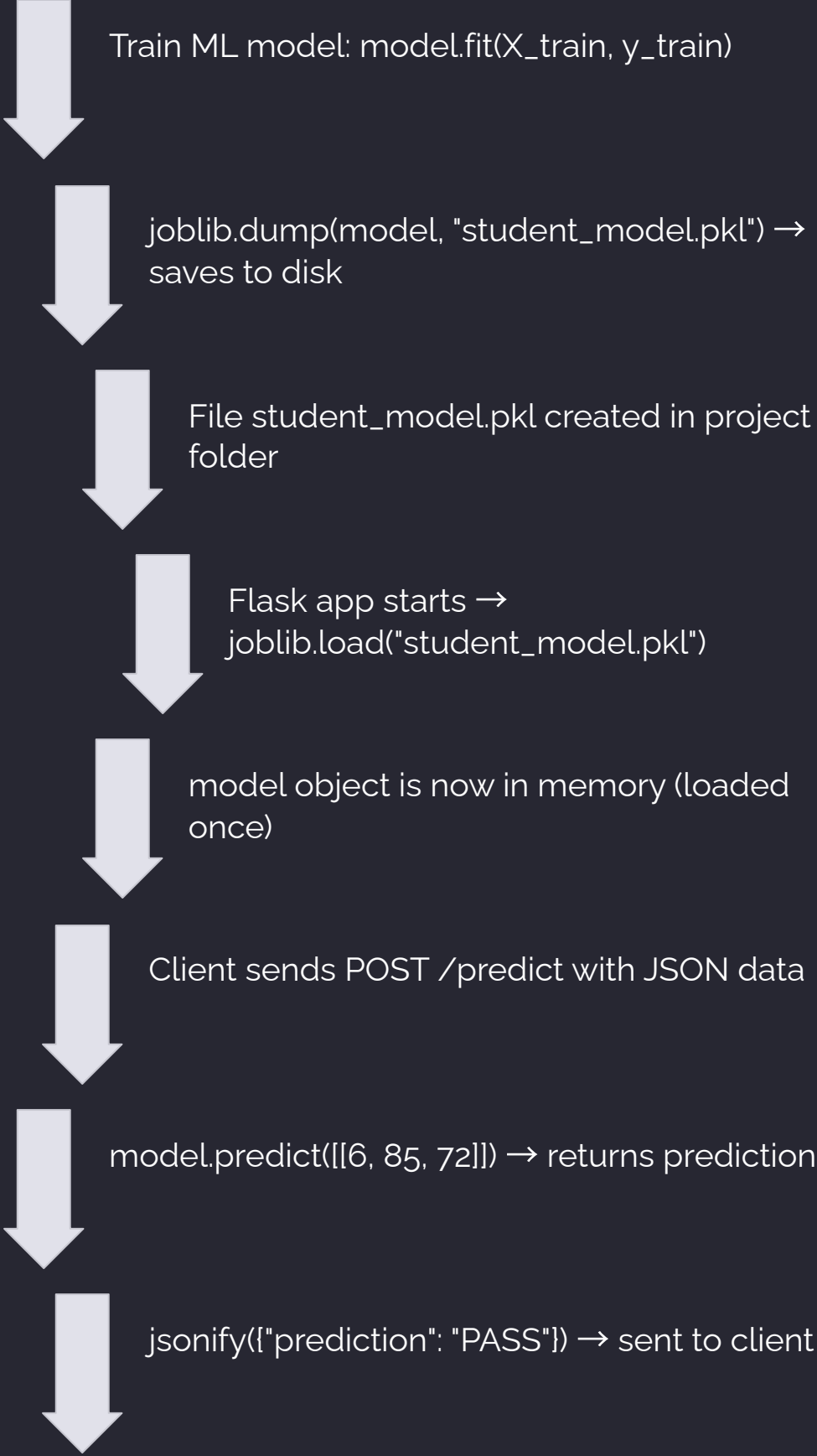
# Save model
with open("model.pkl", "wb") as file:
    pickle.dump(model, file)

# Load model
with open("model.pkl", "rb") as file:
    model = pickle.load(file)
```

joblib vs pickle — Which to Use?

Feature	joblib	pickle
Best for	Scikit-learn models (numpy arrays)	General Python objects
Speed	Faster for large arrays	Slower for large arrays
File size	Smaller .pkl files	Larger .pkl files
Usage	Recommended for ML	General purpose
Import	import joblib	import pickle

The Complete joblib Workflow



Security Warning

Never load untrusted .pkl files

Deserializing a pickle/joblib file from an unknown source can execute malicious code on your server. Only load files you created yourself.

Always save the scaler too!

If you used StandardScaler during training, save it separately: `joblib.dump(scaler, 'scaler.pkl')`. Load and apply it before every prediction.

❏ Rule: `joblib.dump()` during training -> `joblib.load()` in Flask. The model file is the bridge between your training script and your API.

Library 7: requests — Calling APIs from Python

The requests library lets a Python program act as a client and call any REST API. Use it to test your Flask API from Python, or to call external APIs (weather, maps, etc.).

Install & Basic Usage

```
pip install requests

import requests

# --- GET Request ---
response = requests.get(
    "http://127.0.0.1:5000/student"
)
print(response.status_code) # 200
print(response.json())
# {'name': 'Rahul', 'marks': 85}

# --- POST Request (calling /predict) ---
url = "http://127.0.0.1:5000/predict"

payload = {
    "study_hours": 6,
    "attendance": 85,
    "previous_marks": 72
}

response = requests.post(url, json=payload)

print(response.status_code) # 200
print(response.json())
# {'prediction': 'PASS'}
```

Handling Errors Properly

```
try:
    response = requests.post(url, json=payload,
                             timeout=5)
    response.raise_for_status() # raises on 4xx/5xx
    data = response.json()
    print("Prediction:", data["prediction"])

except requests.exceptions.Timeout:
    print("Server took too long to respond")
except requests.exceptions.ConnectionError:
    print("Cannot connect to Flask server")
except requests.exceptions.HTTPError as e:
    print(f"HTTP Error: {e}")
```

Python Client → Flask API Flow

Python script: import requests

Define URL: url = "http://127.0.0.1:5000/predict"

Define payload: payload = {"study_hours": 6, ...}

requests.post(url, json=payload) → sends HTTP POST

Flask receives request → processes → returns JSON

response.status_code → 200 (success) or 400/500 (error)

response.json() → converts JSON string to Python dict

Access result: data["prediction"] → "PASS"

requests vs Postman — When to Use Each

Tool	Best For	Example
Postman	Manual testing, exploring APIs	Click-and-test during development
requests	Automated testing, scripts	test_api.py, batch testing
Browser	GET requests only	Checking /health endpoint
curl	Quick terminal tests	curl -X POST /predict

requests is also used to call external APIs: weather data, Google Maps, payment gateways, SMS services. The same pattern works for all of them.

Library 8 & 9: Flask-RESTful & Pydantic — Advanced API Building

Flask-RESTful provides a class-based structure for building REST APIs. Pydantic validates incoming data types automatically — both are used in larger, production-grade projects.

Flask-RESTful — Class-Based REST API

```
pip install flask-restful

from flask import Flask
from flask_restful import Api, Resource

app = Flask(__name__)
api = Api(app)

# Each Resource = one endpoint
class Student(Resource):

    def get(self):
        return {
            "name": "Rahul",
            "marks": 85
        }

    def post(self):
        return {
            "message": "Student created"
        }, 201

class StudentByld(Resource):

    def get(self, student_id):
        return {
            "id": student_id,
            "name": "Priya",
            "marks": 90
        }

# Register resources with URLs
api.add_resource(Student, "/student")
api.add_resource(StudentByld, "/student/<int:student_id>")

if __name__ == "__main__":
    app.run(debug=True)
```

Flask vs Flask-RESTful

Feature	Flask	Flask-RESTful
Structure	Function-based	Class-based (Resource)
HTTP methods	@app.route(methods=[...])	def get(), def post() methods
Best for	Simple APIs, ML projects	Larger REST APIs
Code style	Simpler, less boilerplate	More organized, scalable

Pydantic — Automatic Data Validation

```
pip install pydantic

from flask import Flask, request, jsonify
from pydantic import BaseModel, ValidationError

app = Flask(__name__)

# Define expected data shape
class StudentInput(BaseModel):
    name: str
    age: int
    study_hours: float
    attendance: float

@app.route("/student", methods=["POST"])
def create_student():
    try:
        # Validate incoming JSON automatically
        data = StudentInput(**request.get_json())

        return jsonify({
            "message": f"Valid! Name: {data.name}",
            "age": data.age
        }), 200

    except ValidationError as e:
        return jsonify({
            "error": e.errors()
        }), 400
```

What Pydantic Catches Automatically

- ### Wrong Type

Client sends age: "twenty" (string) instead of age: 20 (int) → Pydantic raises ValidationError automatically
- ### Missing Field

Client forgets to send "name" → Pydantic returns: field required error with field name
- ### Type Coercion

Client sends study_hours: "6" (string) → Pydantic auto-converts to 6.0 (float) where safe
- Pydantic is the backbone of FastAPI. Learning it with Flask prepares you for FastAPI — the fastest-growing Python API framework in industry.

Complete ML + Flask Project — All Libraries Together

This is the final combined example that uses Flask, request, jsonify, flask-cors, joblib, and numpy together — the exact pattern for a B.Tech ML project.

app.py — Complete Production-Ready API

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import joblib
import numpy as np

# --- Setup ---
app = Flask(__name__)
CORS(app) # Allow frontend to call this API

# Load model & scaler ONCE at startup
model = joblib.load("student_model.pkl")
scaler = joblib.load("student_scaler.pkl")

# --- Routes ---
@app.route("/")
def home():
    return jsonify({
        "message": "Student Prediction API",
        "status": "running"
    })

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200

@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json()

        study_hours = float(data["study_hours"])
        attendance = float(data["attendance"])
        previous_marks = float(data["previous_marks"])

        # Prepare & scale input
        input_data = np.array([
            study_hours,
            attendance,
            previous_marks
        ])
        input_scaled = scaler.transform(input_data)

        # Predict
        prediction = model.predict(input_scaled)[0]
        confidence = model.predict_proba(
            input_scaled)[0]

        result = "PASS" if prediction == 1 else "FAIL"

        return jsonify({
            "prediction": result,
            "confidence": f"{round(float(max(confidence))*100,2)}%"
        }), 200

    except KeyError as e:
        return jsonify({
            "error": f"Missing field: {str(e)}"
        }), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)
```

Step-by-Step Execution Walkthrough

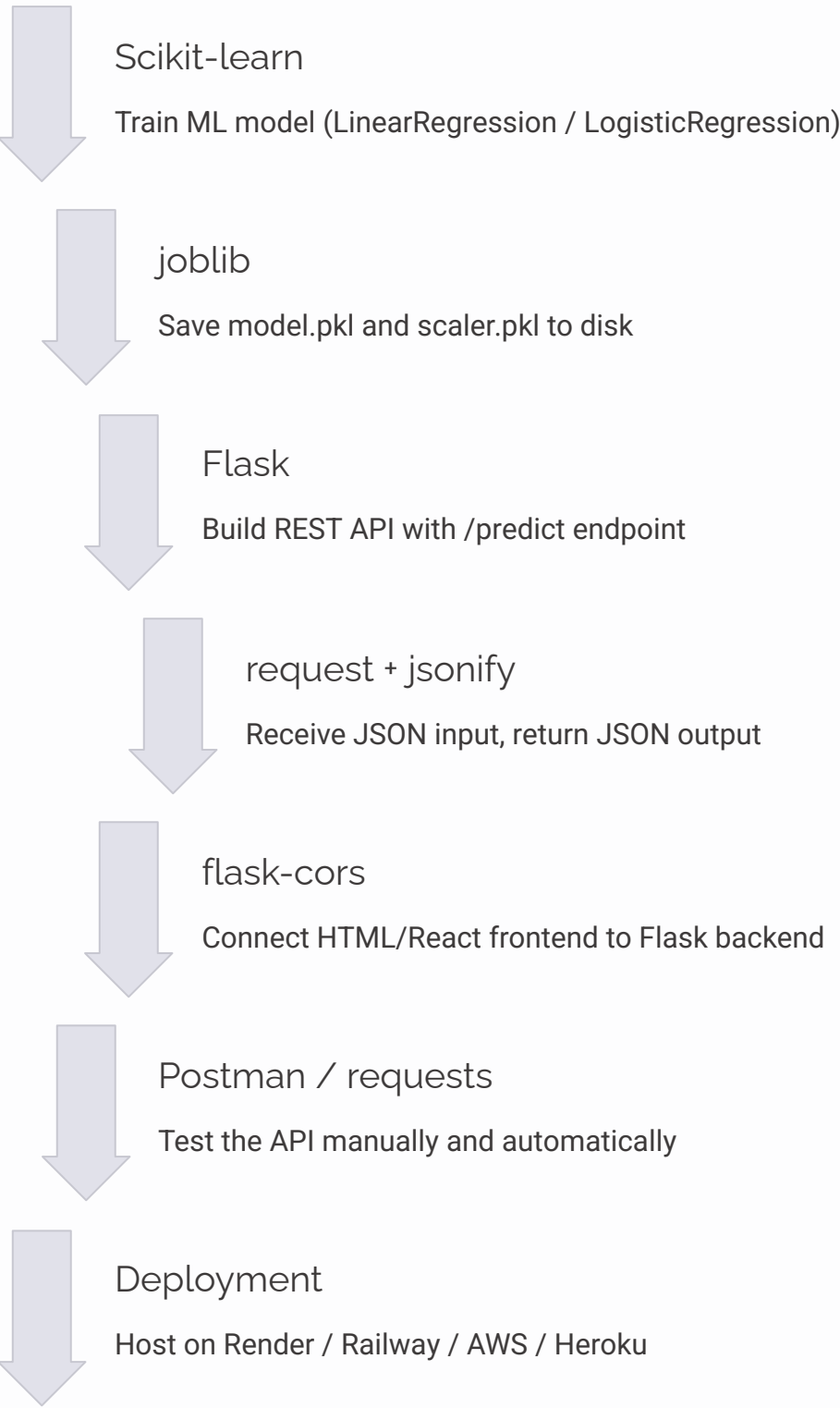
- 01
- CORS(app) — Allows React/HTML frontend on any port to call this API
- 02
- joblib.load() x2 — Model and scaler loaded once at startup (not per request)
- 03
- POST /predict received — Flask routes to predict() function
- 04
- request.get_json() — Parses {"study_hours":6,"attendance":85,"previous_marks":72}
- 05
- float() casting — Ensures numeric types even if client sends strings
- 06
- np.array([[...]]) — Creates 2D array shape (1,3) required by scikit-learn
- 07
- scaler.transform() — Applies same normalization used during training
- 08
- model.predict() — Returns 0 (FAIL) or 1 (PASS)
- 09
- model.predict_proba() — Returns [0.12, 0.88] → confidence = 88%
- 10
- jsonify() — Returns {"prediction":"PASS","confidence":"88%"} with HTTP 200

Library Role Summary

Library	Role in this file
Flask	Creates app, routes, runs server
flask-cors	Allows frontend to call the API
joblib	Loads model.pkl and scaler.pkl
numpy	Formats input as 2D array for model
request	Reads JSON body from POST request
jsonify	Sends JSON response to client

B.Tech ML Project — Full Roadmap & What to Remember

The complete learning path from Python basics to a deployed ML API. This is the industry-standard pipeline every B.Tech student should master.



One-Line Memory Rule for Each Library

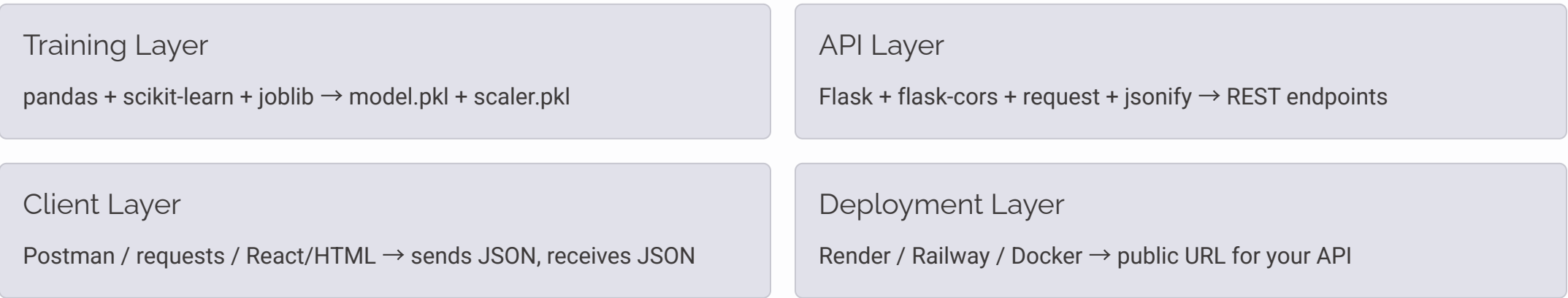
Flask	Creates the API server and routes
request	Reads JSON data sent by the client
jsonify	Sends JSON response back to client
flask-cors	Allows frontend to talk to backend
joblib	Saves and loads the trained ML model
pickle	Serializes any Python object to disk
requests	Calls APIs from a Python script
Flask-RESTful	Class-based REST API structure
pydantic	Validates incoming data types automatically

requirements.txt for Your Project

```
flask==3.0.0
flask-cors==4.0.0
flask-restful==0.3.10
scikit-learn==1.4.0
numpy==1.26.0
pandas==2.2.0
joblib==1.3.2
requests==2.31.0
pydantic==2.5.0
```

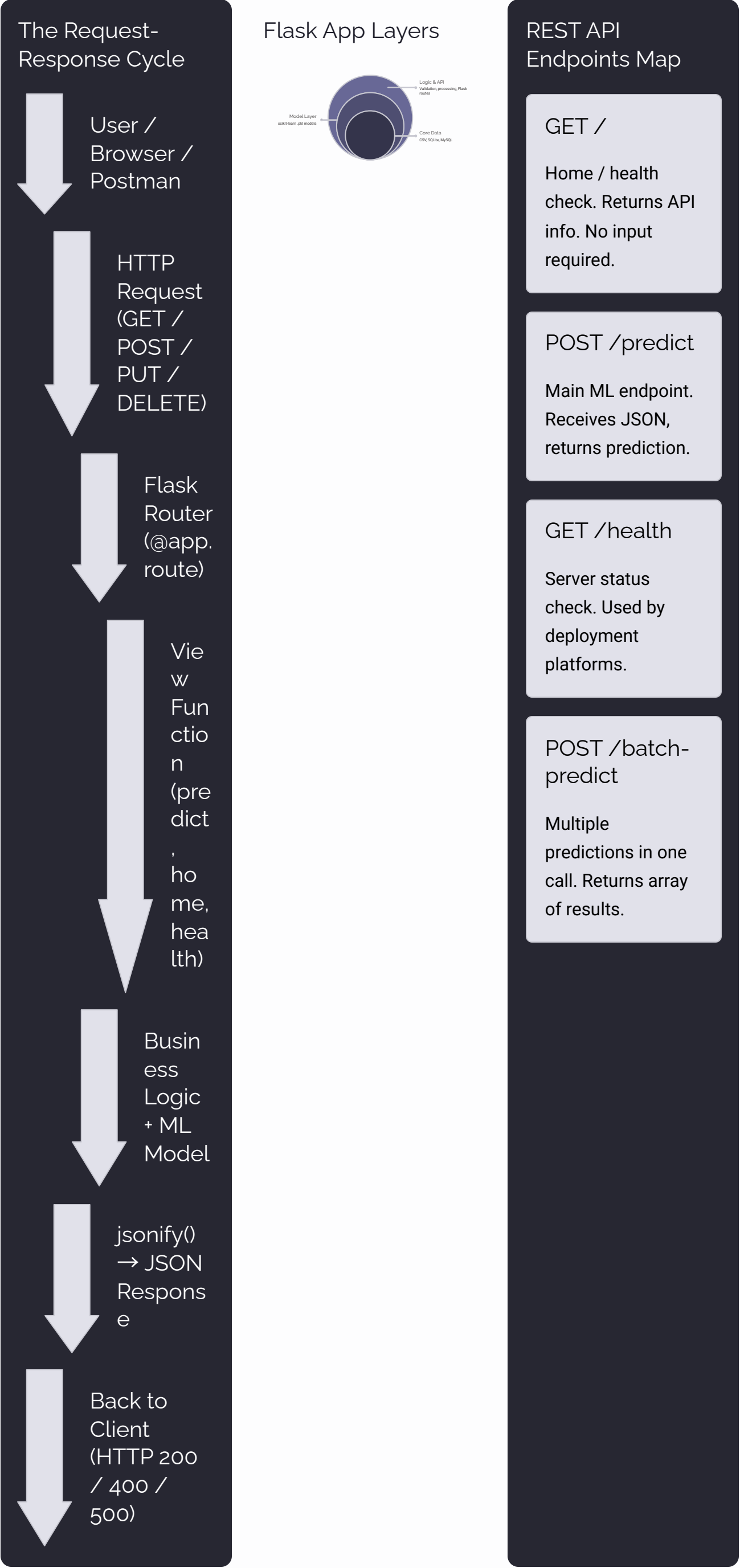
Save this as requirements.txt. Install all at once: `pip install -r requirements.txt`

Final Architecture — The Complete Picture



Flask & RESTful APIs — Visual Architecture Map

Every Flask REST API project follows the same visual pattern. Understanding this architecture is the foundation of all ML web projects.



numpy & pandas — The Data Engine Behind ML APIs

numpy and pandas are the two most important data libraries in Python ML. numpy handles arrays for model input; pandas handles CSV datasets for training.

numpy — Arrays for ML Models

```
import numpy as np

# Scikit-learn needs 2D array input
# Single student: shape (1, 3)
input_data = np.array([[6, 85, 72]])
print(input_data.shape) # (1, 3)

# Multiple students: shape (3, 3)
batch = np.array([
    [6, 85, 72],
    [2, 45, 38],
    [8, 90, 88]
])
print(batch.shape) # (3, 3)

# In Flask /predict endpoint:
features = np.array([
    study_hours, # 6
    attendance, # 85
    prev_marks # 72
])
# shape (1,3) — required by scikit-learn
prediction = model.predict(features)
```

Why np.array([[...]])? — Double Brackets Explained

Single brackets [] → 1D array (WRONG)

np.array([6, 85, 72]) gives shape (3,) — a flat list. scikit-learn rejects this with ValueError.

Double brackets [[]] → 2D array (CORRECT)

np.array([[6, 85, 72]]) gives shape (1, 3) — one row, three columns. scikit-learn accepts this.

pandas — Loading & Preparing Training Data

```
import pandas as pd

# Load dataset from CSV
df = pd.read_csv("students.csv")
print(df.head())
# study_hours attendance prev_marks result
# 0      6      85      72      1
# 1      2      45      38      0

# Select features (X) and target (y)
X = df[["study_hours", "attendance", "prev_marks"]]
y = df["result"]

# Check for missing values
print(df.isnull().sum())

# Basic statistics
print(df.describe())

# Filter: students who passed
passed = df[df["result"] == 1]
print(f"Passed: {len(passed)}")
```

numpy vs pandas — When to Use Each

Task	Use	Why
Model input (predict)	numpy array	scikit-learn requires np.array
Load CSV dataset	pandas DataFrame	Easy column selection & filtering
Feature matrix X	pandas → numpy	df.values → converts to array
Batch processing	numpy	Fast vectorized operations
Data exploration	pandas	.head(), .describe(), .isnull()

📌 Quick rule: Use pandas to load and explore data. Use numpy to feed data into the ML model. They work perfectly together.



Error Handling & HTTP Status Codes in Flask APIs

Proper error handling makes your API professional and safe. Every Flask endpoint must handle bad input, missing fields, and unexpected errors gracefully.

Complete Error Handling Pattern

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Step 1: Check Content-Type
        if not request.is_json:
            return jsonify({
                "error": "Content-Type must be application/json"
            }), 415

        data = request.get_json()

        # Step 2: Check required fields
        required = ["study_hours", "attendance", "prev_marks"]
        for field in required:
            if field not in data:
                return jsonify({
                    "error": f"Missing field: {field}"
                }), 400

        # Step 3: Type conversion
        study_hours = float(data["study_hours"])
        attendance = float(data["attendance"])
        prev_marks = float(data["prev_marks"])

        # Step 4: Range validation
        if not (0 <= study_hours <= 24):
            return jsonify({
                "error": "study_hours must be 0 to 24"
            }), 422
        if not (0 <= attendance <= 100):
            return jsonify({
                "error": "attendance must be 0 to 100"
            }), 422
        if not (0 <= prev_marks <= 100):
            return jsonify({
                "error": "prev_marks must be 0 to 100"
            }), 422

        # Step 5: Predict
        import numpy as np
        features = np.array([[study_hours, attendance, prev_marks]])

        pred = model.predict(features)[0]
        result = "PASS" if pred == 1 else "FAIL"

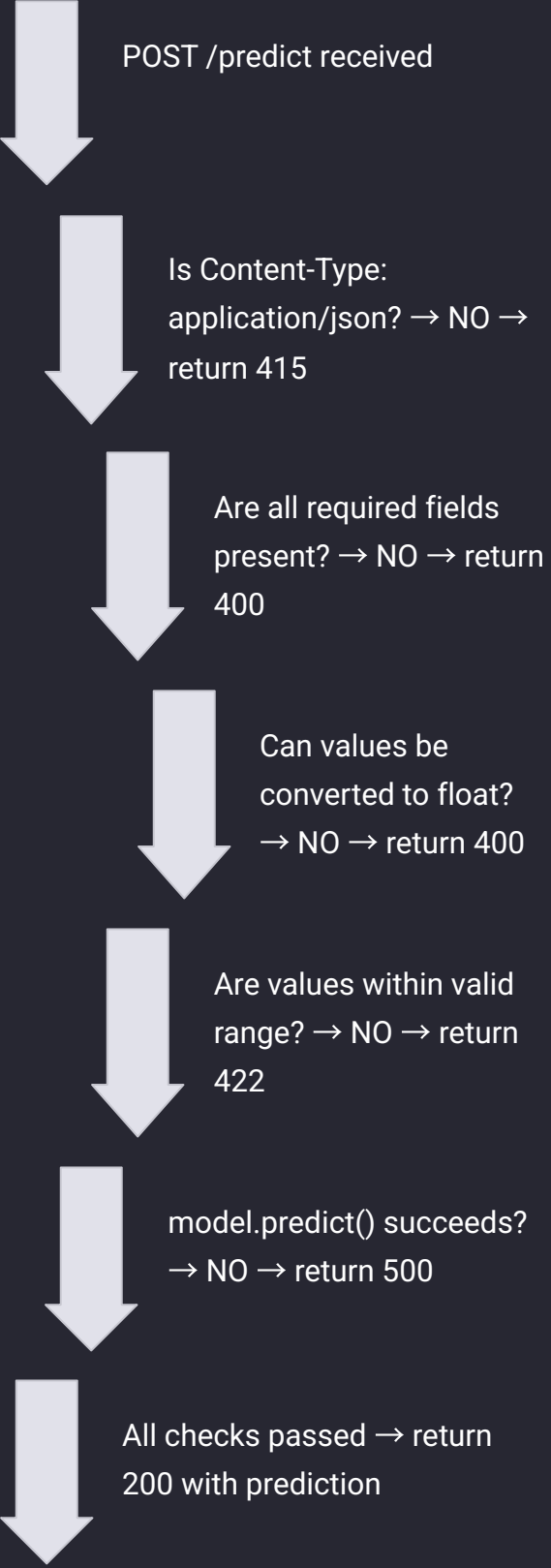
        return jsonify({
            "prediction": result,
            "status": "success"
        }), 200

    except ValueError as e:
        return jsonify({
            "error": f"Invalid data type: {str(e)}"
        }), 400
    except Exception as e:
        return jsonify({
            "error": "Internal server error",
            "detail": str(e)
        }), 500
```

HTTP Status Codes — Complete Reference

Cod e	Name	When to Use in Flask
200	OK	Prediction successful, data returned
201	Create d	New record saved to database
400	Bad Reques t	Missing field, wrong JSON format
404	Not Found	Student ID doesn't exist
415	Unsup ported Media Type	Content-Type is not application/json
422	Unproc essabl e Entity	Field present but value out of valid range
500	Internal Server Error	Unexpected crash, model load failure

Error Handling Flowchart



❏ Golden rule: Validate BEFORE you predict. Never pass unvalidated user input directly to your ML model.

Database with Flask — SQLite & Flask-SQLAlchemy

Store student predictions in a database so you can retrieve history, generate reports, and build a complete CRUD API. SQLite requires zero setup — perfect for B.Tech projects.

Setup & Models

```
pip install flask flask-sqlalchemy

from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
import joblib, numpy as np

app = Flask(__name__)

# SQLite database file
app.config["SQLALCHEMY_DATABASE_URI"] = \
    "sqlite:///students.db"
db = SQLAlchemy(app)

# Database Table Model
class Prediction(db.Model):
    id = db.Column(db.Integer, primary_key=True)

    study_hours = db.Column(db.Float)
    attendance = db.Column(db.Float)
    prev_marks = db.Column(db.Float)
    result = db.Column(db.String(10))

# Create tables
with app.app_context():
    db.create_all()

model = joblib.load("student_model.pkl")
```

POST /predict — Save to Database

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    sh = float(data["study_hours"])
    att = float(data["attendance"])
    pm = float(data["prev_marks"])

    features = np.array([[sh, att, pm]])
    pred = model.predict(features)[0]
    result = "PASS" if pred == 1 else "FAIL"

    # Save to database
    record = Prediction(
        study_hours = sh,
        attendance = att,
        prev_marks = pm,
        result = result
    )
    db.session.add(record)
    db.session.commit()

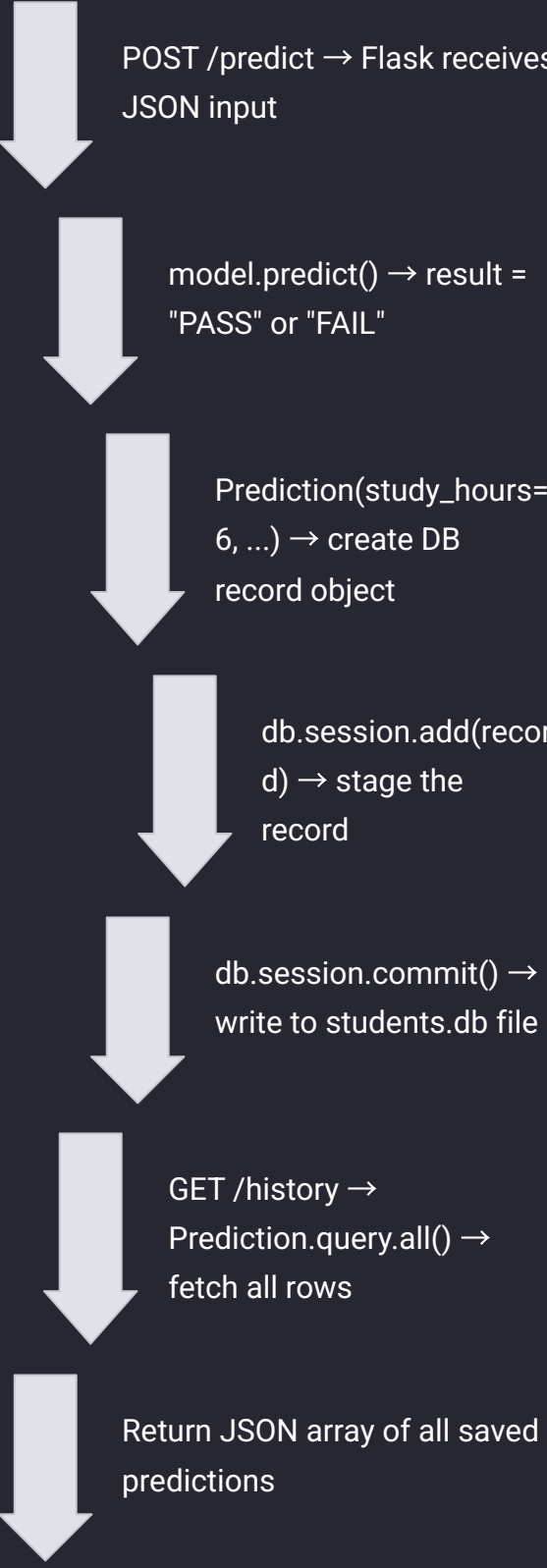
    return jsonify({
        "prediction": result,
        "saved_id": record.id
    }), 200
```

GET /history — Retrieve All Predictions

```
@app.route("/history", methods=["GET"])
def history():
    records = Prediction.query.all()
    results = []
    for r in records:
        results.append({
            "id": r.id,
            "study_hours": r.study_hours,
            "attendance": r.attendance,
            "prev_marks": r.prev_marks,
            "result": r.result
        })
    return jsonify({
        "total": len(results),
        "records": results
    }), 200

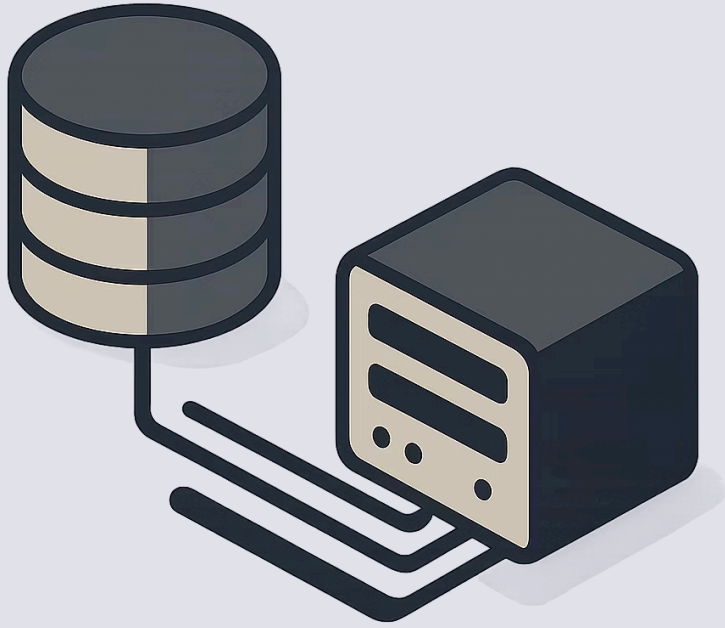
@app.route("/history/<int:id>", methods=["DELETE"])
def delete_record(id):
    record = Prediction.query.get_or_404(id)
    db.session.delete(record)
    db.session.commit()
    return jsonify({
        "message": f"Record {id} deleted"
    }), 200
```

Database Flow Diagram



CRUD Endpoints Summary

Method + URL	Action	Returns
POST /predict	Predict + save to DB	prediction + saved_id
GET /history	Get all predictions	array of records
GET /history/5	Get one record by ID	single record
DELETE /history/5	Delete one record	success message



Frontend Integration — HTML Form + Flask API

Connect a simple HTML form to your Flask /predict API using JavaScript fetch(). No React needed — this works in any browser and is perfect for B.Tech project demos.

index.html — Complete Frontend Form

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Prediction</title>
  <style>
    body { font-family: Arial; max-width: 500px; margin: 50px auto; padding: 20px; }
    input { width: 100%; padding: 8px; margin: 5px 0 15px; box-sizing: border-box; }
    button { background: #007bff; color: white; padding: 10px 20px; border: none; cursor: pointer; border-radius: 4px; }
    #result { margin-top: 20px; padding: 15px; border-radius: 4px; font-size: 18px; }
    .pass { background: #d4edda; color: #155724; }
    .fail { background: #f8d7da; color: #721c24; }
  </style>
</head>
<body>
  <h2>Student Performance Predictor</h2>

  <label>Study Hours (0-24):</label>
  <input type="number" id="study_hours" placeholder="e.g. 6">

  <label>Attendance % (0-100):</label>
  <input type="number" id="attendance" placeholder="e.g. 85">

  <label>Previous Marks (0-100):</label>
  <input type="number" id="prev_marks" placeholder="e.g. 72">

  <button onclick="predict()">Predict Result</button>

  <div id="result"></div>

  <script>
    async function predict() {
      const payload = {
        study_hours:
document.getElementById(
  "study_hours").value,
        attendance:
document.getElementById(
  "attendance").value,
        prev_marks:
document.getElementById(
  "prev_marks").value
      };

      try {
        const response = await fetch(

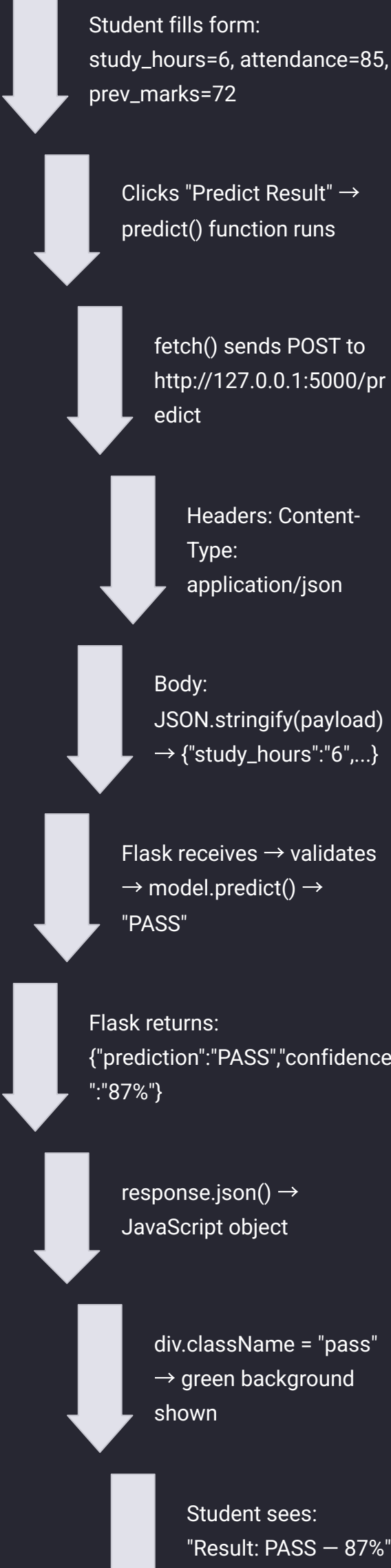
"http://127.0.0.1:5000/predict", {
          method: "POST",
          headers: {
            "Content-Type":
"application/json"
          },
          body:
JSON.stringify(payload)
        }
      );

      const data = await
response.json();
      const div =
document.getElementById("result"
);

      if (data.prediction === "PASS")
      {
        div.className = "pass";
        div.innerHTML = "Result:
PASS — " +
          data.confidence;
      } else {
        div.className = "fail";
        div.innerHTML = "Result:
FAIL — " +
          data.confidence;
      }
    } catch (error) {

document.getElementById("result"
)
      .innerHTML = "Error: " +
error.message;
    }
  </script>
</body>
</html>
```

HTML → Flask → HTML Flow



Key JavaScript Concepts

"async / await"

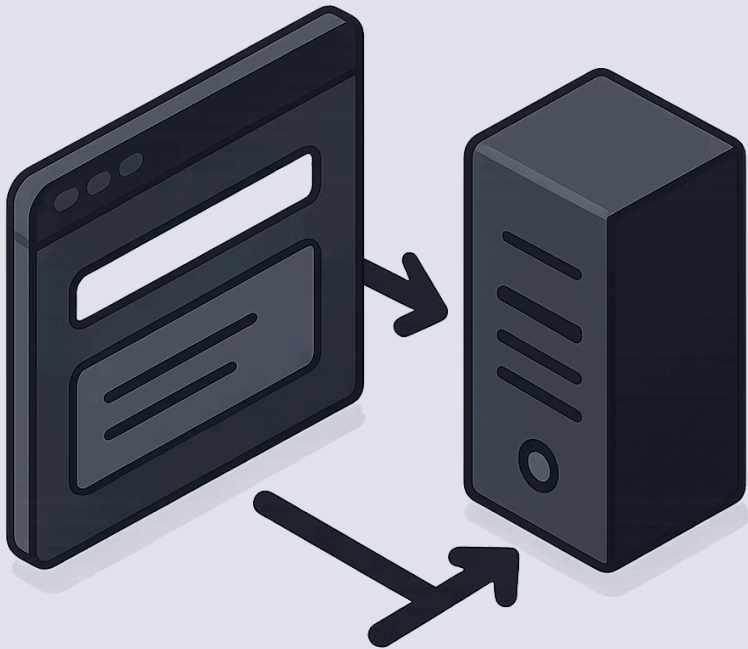
Makes the fetch() call wait for Flask to respond before reading the result. Without this, data would be undefined.

"JSON.stringify()"

Converts JavaScript object to JSON string for sending. Flask's request.get_json() reads this string.

"response.json()"

Converts Flask's JSON response string back into a JavaScript object so you can access data.prediction.



❑ CORS must be enabled in Flask (CORS(app)) for this HTML page to call the API. Without it, the browser will block the fetch() request.

Deployment — Taking Your Flask API Live

Deployment means making your Flask API accessible on the internet with a public URL. Render.com is the easiest free platform for B.Tech student projects.

Step-by-Step: Deploy to Render (Free)

- 01
- Create these 3 files in your project folder
- 02
- Push project to GitHub (git init → git push)
- 03
- Go to render.com → Sign up with GitHub
- 04
- Click "New Web Service" → Connect your GitHub repo
- 05
- Set Build Command: pip install -r requirements.txt
- 06
- Set Start Command: gunicorn app:app
- 07
- Click Deploy → Wait 2-3 minutes
- 08
- Your API is live at: <https://your-app.onrender.com>

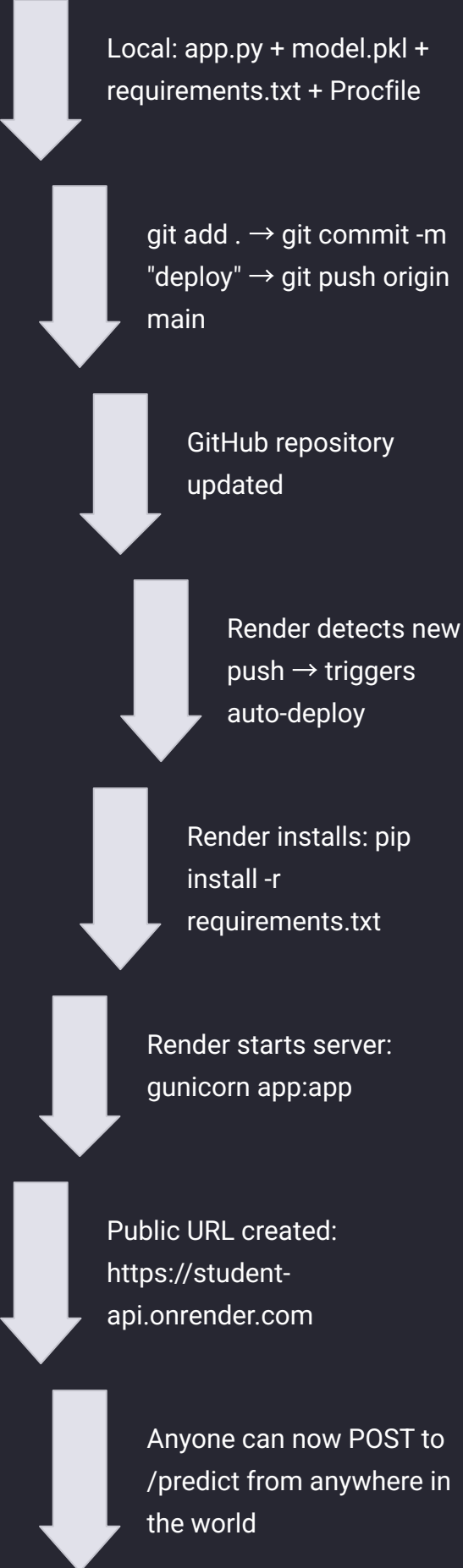
Required Files for Deployment

```
flask==3.0.0
flask-cors==4.0.0
scikit-learn==1.4.0
numpy==1.26.0
joblib==1.3.2
gunicorn==21.2.0
```

web: gunicorn app:app

python-3.11.0

Deployment Flow Diagram



Platform Comparison

Platform	Free Tier	Best For	URL
Render	Yes (sleeps after 15min)	Student projects	render.com
Railway	Yes (\$5 credit)	Fast deploys	railway.app
PythonAnywhere	Yes (limited)	Pure Python APIs	pythonanywhere.com
AWS / GCP	Free tier (12 months)	Production apps	aws.amazon.com

gunicorn vs app.run()

app.run(debug=True) — Development Only

Flask's built-in server handles one request at a time. Never use in production. Only for local testing.

gunicorn app:app — Production Server

Gunicorn handles multiple requests simultaneously. Required by all deployment platforms. app:app means file=app.py, object=app.

